



Universidade Federal do Cariri
Ciência da Computação
Juazeiro do Norte, CE
2025

Computação Gráfica

Material de Apoio XII - Superfícies Visíveis

Monitor(a): Felipe Coêlho
Profª Responsável: Luana Batista



Sumário

1.	Introdução	3
2.	Back-Face Culling	4
2.1.	Conceitos	4
2.2.	Funcionamento	4
2.3.	Limitações	5
3.	Z-Buffer	5
3.1.	Conceitos	5
3.2.	Funcionamento	7
3.3.	Etapas	7
3.4.	Vantagens e Desvantagens	7
4.	Implementação OpenGL	8
4.1.	Back-Face Culling	8
4.2.	Z-Buffer	9



1 - Introdução

No processo de renderização de polígonos, buscamos otimizar o desempenho renderizando apenas as faces que estão visíveis do ponto de vista da câmera. Para isso, há várias técnicas capazes de determinar quais superfícies devem ser exibidas (ou remover aquelas ocultas), e essas técnicas variam conforme a complexidade da cena, o tipo de geometria dos objetos (convexos, côncavos, etc.), os recursos computacionais disponíveis, entre outros fatores.

Os métodos disponíveis para detecção de superfícies visíveis podem ser agrupados em duas categorias principais: as **técnicas baseadas no espaço do objeto**, que avaliam objetos ou suas partes (como faces ou arestas) entre si para decidir o que está visível; e as **técnicas baseadas no espaço da imagem**, que analisam individualmente cada pixel da imagem projetada para determinar qual superfície deve ser exibida.

Neste material, vamos abordar dois algoritmos representativos dessas abordagens: o **Back-face culling**, uma técnica simples e eficiente do espaço de objeto usada para eliminar automaticamente as faces traseiras de um poliedro, e o **Z-buffer**, um método amplamente adotado do espaço da imagem, que determina a visibilidade comparando as profundidades das superfícies em cada pixel do plano de projeção.

2 - Back-Face Culling

2.1 Conceitos

O **Back-Face Culling** é uma técnica simples que descarta as **faces traseiras** de um objeto antes do processo de rasterização. Assim, quando as faces fazem parte de um objeto sólido, como um poliedro, não é preciso desenhar as que estão voltadas para trás, pois não estarão visíveis na cena. Dessa forma, apenas as faces frontais, voltadas para a câmera, precisam ser renderizadas, economizando processamento.

2.2 Funcionamento (Figura 1)

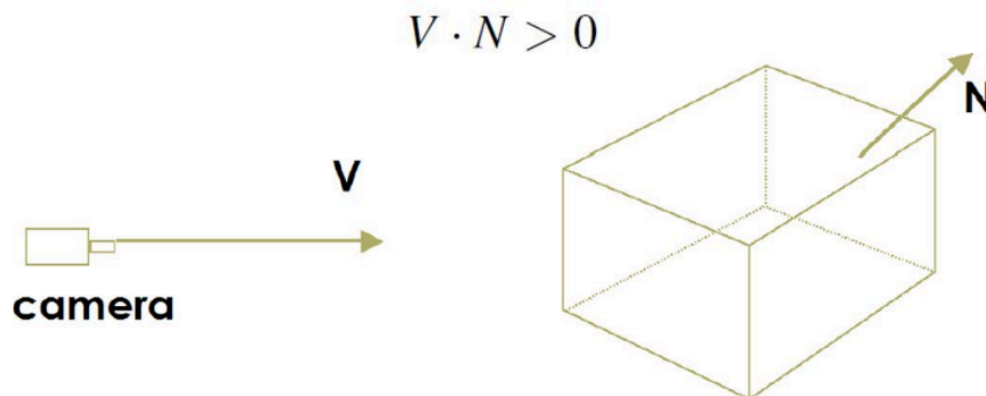


Figura 1

- A face é considerada **não visível** se o ângulo entre a normal da face (**N**) e o vetor de visão (**V**) for maior que 90°.
- Em OpenGL, isso pode ser automatizado utilizando o **sentido dos vértices** (ordem horária/anti-horária) para identificar o “lado de trás”.

2.3 Limitações

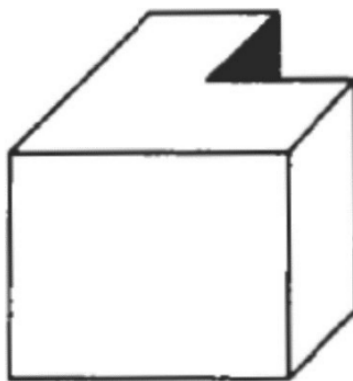


Figura 2

- Não funciona corretamente para objetos **côncavos** ou **transparentes**, pois pode haver faces parcialmente visíveis que o algoritmo considera como inteiramente visíveis. Assumindo, assim, que os objetos são sólidos e **convexos (Figura 2)**.

3 - Algoritmo Z-Buffer

3.1 Conceitos

Em uma cena 3D, várias superfícies podem estar posicionadas umas atrás das outras. O objetivo é desenhar apenas as partes visíveis (**Figura 3**) (ou totalmente expostas) dessas superfícies.

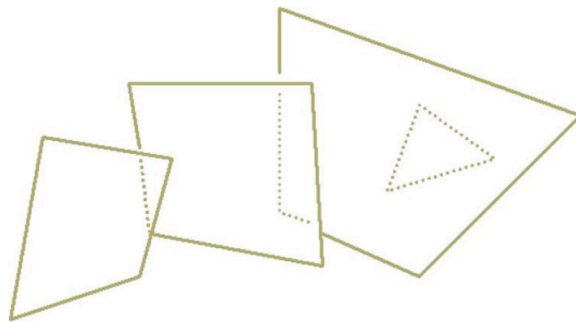


Figura 3

Surge então outro desafio: o tratamento de superfícies escondidas. O algoritmo precisa lidar com situações onde partes de objetos estão ocultas por outros, garantindo que apenas as regiões visíveis sejam renderizadas na tela.

A solução está no algoritmo **Z-buffer**, que resolve a visibilidade comparando a **profundidade (coordenada z)** de cada pixel renderizado. É um método de **espaço de imagem**, amplamente usado por ser simples e eficaz.

Imagine que as faces já foram projetadas e possuem suas coordenadas de profundidade (z) registradas — com valores normalizados entre 0 (plano mais próximo) e 1 (plano mais distante). Para cada pixel (x, y), o objetivo é identificar qual face está mais perto da câmera, ou seja, qual possui o menor valor de z (**Figura 4**).

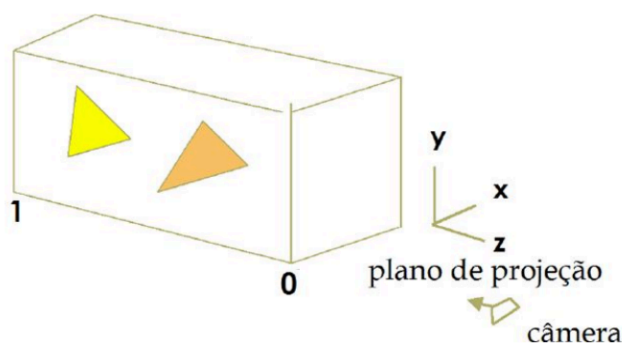


Figura 4

3.2 Funcionamento

- Cada pixel no **framebuffer** possui um valor de cor (RGB).
- O **z-buffer** armazena a profundidade (z) de cada pixel.
- A cada nova superfície desenhada:
 - Se o novo z for menor (mais próximo da câmera), o pixel é atualizado.
 - Caso contrário, é ignorado.

3.3 Etapas

1. Inicialização:

- Z-buffer recebe valor máximo (1.0).
- Framebuffer recebe a cor de fundo.

2. Processamento de faces:

- Faces são rasterizadas.
- A profundidade é comparada com o valor atual do z-buffer.
- Se for mais próxima, atualiza o z-buffer e a cor no framebuffer.

3.4 Vantagens e Desvantagens

Vantagens:

- Fácil de implementar e frequentemente suportado diretamente por hardware gráfico;
- Permite desenhar os objetos em qualquer sequência;
- As GPUs realizam otimizações específicas para operações envolvendo o Z-buffer.

Desvantagens:

- Requer uma quantidade significativa de memória — por exemplo, em uma tela de 1280×1024 são necessários aproximadamente 1,3 milhões de posições no buffer, além de um depth buffer do mesmo tamanho;
- Pode haver cálculos redundantes devido à ausência de uma ordem de desenho otimizada.

4 - Implementação OpenGL

4.1 Back-Face Culling

Para ativar o **back-face culling** no **OpenGL**, é necessário, primeiramente, habilitar o recurso utilizando a função **glEnable(GL_CULL_FACE)**.

Em seguida, deve-se especificar qual face será descartada por meio da função **glCullFace(GL_BACK)**, que indica a remoção das faces traseiras.

Além disso, é preciso informar ao **OpenGL** qual é o sentido considerado como “frontal” para as faces do objeto. Essa definição é feita com a função **glFrontFace**, que recebe como parâmetro **GL_CCW** (*counter-clockwise*) quando se deseja considerar que as faces com vértices ordenados no sentido anti-horário estão voltadas para a câmera. Esse é o padrão mais comum, especialmente em bibliotecas como **GLUT**.

Vale destacar que, caso não sejam explicitamente fornecidos os vetores normais das faces, o **OpenGL** utilizará o sentido dos vértices como critério para determinar se a face é frontal ou traseira. Dessa forma, a correta ordenação dos vértices é essencial para garantir que o **back-face culling** funcione adequadamente.

4.2 Z-Buffer

Para utilizar o **z-buffer** no **OpenGL** com **GLUT**, primeiramente, no momento da inicialização da janela, é necessário solicitar explicitamente a criação de um **depth buffer**. Isso é feito por meio da função **glutInitDisplayMode**, combinando as flags **GLUT_SINGLE** ou **GLUT_DOUBLE**, **GLUT_RGB** e **GLUT_DEPTH**. A flag **GLUT_DEPTH** garante a alocação do buffer de profundidade.

Por exemplo:

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
```

Em seguida, na função de inicialização da aplicação (função normalmente chamada de *init*), é necessária para ativar o uso do **z-buffer** por meio da chamada **glEnable(GL_DEPTH_TEST)**. Esta função informa ao **OpenGL** que a profundidade dos objetos deve ser considerada no processo de rasterização, e que a renderização de um fragmento só ocorrerá se sua profundidade for menor do que a já registrada no buffer.

Por fim, na função de exibição da cena (*display*), é fundamental que o **z-buffer** seja **limpo a cada novo quadro**, juntamente com o buffer de cor. Essa limpeza garante que os dados de profundidade da renderização anterior não interfiram na próxima. A função utilizada para isso é:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Ela limpa tanto o **framebuffer** (cores da imagem) quanto o **z-buffer** (profundidade dos pixels), garantindo uma renderização correta e atualizada a cada novo ciclo de desenho.